UNIVERSIDAD DE LOS LLANOS — INGENIERÍA DE SISTEMAS

Procesos Estocásticos (Código 603504) | Proyecto Final HMM 2D/3D

ESPECIFICACIÓN TÉCNICA ACTUALIZADA DEL SISTEMA (SPEC)

Reconocedor y Clasificador de Caracteres en Tiempo Real 2D/3D mediante Cadenas Ocultas de Markov

Docente: Yerson Porras (`yerson.porras@unillanos.edu.co`)	**Versión:** Actualizada al estado final del proyecto (2026-11-12)
Enunciado base: `docs/enunciado-proyecto-final-reconocedor-hmm.pdf`	**Ponderación:** 90% del Tercer Corte | Defensa Oral + Código + ZIP

1. Propósito, Alcance y Restricciones Fundamentales

El sistema implementa desde cero una aplicación de escritorio en Python 3.12 (`PySide6` + `PyQtGraph/PyOpenGL`) capaz de capturar, discretizar, clasificar y autocompletar en tiempo real las **27 letras minúsculas del alfabeto español**:

$$C = \{ a, b, c, d, e, f, g, h, i, j, k, l, m, n, ñ, o, p, q, r, s, t, u, v, w, x, y, z \}$$

(Sin dígrafos `"ch"` ni `"ll"` y sin vocales con tilde). Toda la lógica matemática de los Modelos Ocultos de Markov —incluyendo `Forward` escalado, `Backward`, `Viterbi` en espacio logarítmico, entrenamiento `Baum-Welch`, proyección de trayectoria futura y clasificación bayesiana— está implementada de forma **100% nativa utilizando exclusivamente NumPy**, cumpliendo la prohibición expresa de librerías externas (`hmmlearn`, `scikit-learn`, `scipy.stats`, PyTorch o TensorFlow) verificada en `requirements.txt`.

2. Requerimientos Funcionales de la Interfaz Gráfica (GUI)

La interfaz gráfica está orquestada por `HMMRecognizerApp` en `main.py` con barra de navegación lateral hacia cuatro vistas desacopladas de cualquier lógica de entrenamiento:

2.1 Ventana A — Menú Principal de Inicio (`gui/MainMenu.py`)

- Pantalla minimalista "*Elige un modo*" con tarjetas interactivas vectoriales para acceder directamente a **Modo 2D** y **Modo 3D**, además de la barra lateral persistente con botones *Inicio*, *Modo 2D*, *Modo 3D* y *Guía de letras*.

2.2 Ventana B — Reconocedor y Visualizador 2D (`gui/View2D.py`)

- **Lienzo interactivo (`RecognizerCanvas2D`):** captura trazos y sub-trazos continuos con clic izquierdo dentro de un recuadro guía de `250 × 250 px`. Muestra saltos entre sub-trazos en línea punteada ámbar y el autocompletado futuro en verde punteado.
- **Escala adaptativa en vivo vs trazo finalizado:** mientras el usuario arrastra el ratón (`is_complete=False`), extrae con piso `expected_letter_size = 250 px` para preservar el prefijo parcial; al soltar el clic (si la extensión es ≥ 35 px), evalúa con `realtime=False` normalizando la letra completa a $[0, 1]^2$ y acota la proyección futura si Viterbi ya alcanzó el estado final `N - 1`.
- **Controles:** botones *Reiniciar captura* y *Limpiar lienzo*, y panel derecho `RankingPanel`.

2.3 Ventana C — Reconocedor y Visualizador 3D Esférico (`gui/View3D.py`)

- **Esfera interactiva OpenGL (`SphereCanvas3D`):** renderizado 3D con malla sombreada, paralelos/meridianos y un **recuadro guía ecuatorial azul de ±48°** centrado frente a la cámara que cubre la cara visible de la esfera.
- **Interacción y proyección geodésica:** intersección exacta rayo-esfera (`intersect_ray_sphere`) para trazar directamente sobre la superficie, densidad visual mediante interpolación SLERP, rotación con clic derecho o modo *Girar esfera*, zoom con rueda y botón *Centrar esfera*, además de *Reiniciar captura* y *Limpiar lienzo*.

2.4 Panel de Clasificación (`gui/RankingPanel.py`) y Guía de Letras (`gui/LetterGuide.py`)

- **Panel Derecho (`RankingPanel`):** ordena descendentemente las 27 clases en vivo según $P(c \mid O_{1:t}, g)$ mostrando su glifo manuscrito canónico (`data/canonical_glyphs.json`), barra de progreso y porcentaje.
- **Guía de Letras (`LetterGuideView`):** muestra para cada una de las 27 letras los sub-trazos exactos del dataset (incluyendo letras de dos trazos como `d` = `c` + palo vertical, `i`, `j`, `ñ`, `t`, `x`, `f`, `k`), con círculos numerados de inicio (`1`, `2`), flechas de dirección, animación paso a paso (pausa, velocidad y reinicio) y práctica directa con ratón.

3. Modelado Matemático, Discretización y Clasificación HMM

3.1 Discretización Geométrica 2D y Esférica 3D (`core/FeatureExtractor.py`)

- **Alfabeto discreto de observaciones:** $|V| = K + 4 = 20$ símbolos, con $K = 16$ sectores angulares

centrados (`0` = Este, sentido antihorario) y 4 símbolos cardinales de salto con pluma levantada (`→, ↑, ←, ↓`).

- **Invariancia a escala y velocidad en 2D (`FeatureExtractor2D`):** inversión del eje `y` de pantalla, normalización, suavizado simétrico por promedio móvil (`w = 3`) y remuestreo por longitud de arco constante `resample_step = 0.05`.
- **Geometría diferencial sobre S^2 en 3D (`FeatureExtractor3D`):** rotación al marco centrado en la cámara $R_{cam} \in SO(3)$ para situar el dibujo en el ecuador frontal ($\theta = \pi/2, \phi = 0$) lejos de los polos, conversión $(x, y, z) \to (\theta, \phi)$ con desenvolvimiento de ϕ, remuestreo geodésico por SLERP y cálculo del rumbo métrico en el plano tangente local:

$$\psi_m = \operatorname{atan2}(-d\theta_m,\ d\varphi_m \cdot \sin(\bar{\theta}_m))$$

- **Descriptor morfológico global $g \in \mathbb{R}^7$:** $g = [\log(w/h),\ x'_{ini},\ y'_{ini},\ x'_{fin},\ y'_{fin},\ \bar{x}',\ \bar{y}']^T$.

3.2 Motor HMM Bakis, Entrenamiento y Clasificación Bayesiana (`core/HMMEngine.py`)

- **Topología Bakis izquierda-derecha:** 27 modelos independientes $\lambda_c = (A_c, B_c, \pi_c)$ con $N \in [4, 11]$ estados definidos en `config/config.toml`, estado inicial fijo $\pi = [1, 0, \ldots, 0]^T$ y transiciones permitidas $j \in \{i, i+1, i+2\}$.
- **Algoritmos nativos:** Forward escalado (c_t para evitar underflow, $\log P(O|\lambda) = -\Sigma \log c_t$), Backward escalado, Viterbi en log-space, Baum-Welch multisecuencia con suavizado circular angular (`0.78` central + `0.11` en sectores vecinos ± 1) y proyección futura basada en la esperanza geométrica de permanencia $E[d_j] = 1/(1 - a_{jj})$ (en 3D mediante el mapa exponencial sobre S^2).
- **Clasificación conjunta (`HMMClassifier.classify`):** combina secuencia, duración esperada, morfología 7D y conteo de saltos entre sub-trazos, normalizado mediante Log-Sum-Exp sobre las 27 clases:

$$\text{Score}(c) = \log P(O_{1:t} \mid \lambda_c) - P_{dur}(t;\ \mu_c, \sigma_c) - P_{shape}(g) - P_{jump}(J_{obs};\ J_c)$$

4. Arquitectura Limpia, Dataset, Modelos y Verificación

Módulo / Carpeta	Responsabilidad Técnica Verificada
`config/config.toml` `core/config.py`	Única fuente de configuración validada: 27 letras minúsculas, alias `ñ → enie`, hiperparámetros de extracción y tabla de estados `N` por carácter.
`core/HMMEngine.py`	Implementación pura en NumPy de `HMM`, `HMMClassifier`, inicialización Bakis segmentaria y serialización JSON.

`core/FeatureExtractor.py`	Extractores `FeatureExtractor2D` y `FeatureExtractor3D`, descriptor $g \in \mathbb{R}^7$, proyección 2D/3D y ray-casting esférico.
`training/`	Pipeline offline desacoplado de la GUI: `augment.py` (carga de dataset preservando puntos de 1 clic y proyección 3D), `train.py` (entrenamiento unificado 2D/3D) y `evaluate.py` (comparativa `K=8` vs `K=16` en prefijos `30%, 50%, 70%, 100%`).
`data/` y `models/`	`glifo-dataset-2026-11-12.json` (2,560 muestras en 27 clases), `canonical_glyphs.json` (27 plantillas canónicas) y 54 modelos entrenados (`2d_*.json` y `3d_*.json`).
`gui/` y `main.py`	Aplicación de escritorio en 4 vistas (`MainMenu`, `View2D`, `View3D`, `LetterGuide`) y `RankingPanel`; sin dependencias hacia `training/`.
`tests/` y `docs/`	Suite de **33 pruebas automatizadas** (`pytest -q`) y documentación matemática/ arquitectónica para sustentación (`ecuaciones.md`, `AUDITORIA_ARQUITECTURA.md`, `SUSTENTACION_MATEMATICA.md`).

Criterio de verificación automática: La prueba `test_gui_has_no_training_or_capture_links` garantiza en cada ejecución que ni `main.py` ni ningún módulo de `gui/` importe o invoque entrenamiento en tiempo de ejecución, cumpliendo estrictamente el §4.1 del enunciado.